Process-Related Functions in C
Quick Overview

`system`, `fork`, `exec`, `getpid`, `getppid`, `wait`

Paulo Pedreiras, Pedro Fonseca, DETI/IT/UA

Oct/2026

Overview

These functions are commonly used in C programs on Unix-like systems such as Linux.
They are not all part of standard ISO C. Most of them belong to POSIX.

- `system()`: runs a shell command.
- `fork()`: creates a new process.
- `exec()`: replaces the current process image.
- `getpid()`: returns the current process ID.
- `getppid()`: returns the parent process ID.
- `wait()`, `waitpid()`: wait for child processes to finish.

Typical headers:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/wait.h>
```

system()

The function `system()` executes a command using the system shell.

```c
#include <stdlib.h>

int system(const char *command);
```

Example:

```c
int status = system("ls -l");

if (status == -1) {
    perror("system");
}
```

- Simple way to run external commands.
- Internally, it usually creates a new process and invokes a shell.
- The command is interpreted by the shell, so shell features work.
- Be careful with user-provided strings: they may cause command injection.

fork()

The function `fork()` creates a new process.

```
1 #include <unistd.h>
2
3 pid_t fork(void);
```

After `fork()`, there are two processes:

- The parent process.
- The child process.

Return value:

- In the child: `fork()` returns `0`.
- In the parent: `fork()` returns the child's PID.
- On error: `fork()` returns `-1`.

fork()

Example:

```
1 pid_t pid = fork();
2
3 if (pid == -1) {
4     perror("fork");
5 } else if (pid == 0) {
6     printf("I am the child\n");
7 } else {
8     printf("I am the parent, child PID = %d\n", pid);
9 }
```

getpid() and getppid()

`getpid()` returns the process ID of the current process.
`getppid()` returns the process ID of the parent process.

```c
#include <unistd.h>

pid_t getpid(void);
pid_t getppid(void);
```

getpid() and getppid()

Example:

```
1 printf("My PID is %d\n", getpid());
2 printf("My parent's PID is %d\n", getppid());
```

Useful for understanding parent-child relationships:

```
1 pid_t pid = fork();
2
3 if (pid == 0) {
4     printf("Child:  PID = %d, PPID = %d\n",
5             getpid(), getppid());
6 } else {
7     printf("Parent: PID = %d, child PID = %d\n",
8             getpid(), pid);
9 }
```

The exec Family

The exec family replaces the current process image with a new program.
Common variants include:

- `execl()`, `execv()`
- `execlp()`, `execvp()`
- `execle()`, `execve()`

Example using `execlp()`:

```c
#include <unistd.h>

execlp("ls", "ls", "-l", NULL);

perror("execlp");  /* only reached if exec fails */
```

Important idea:

- If exec succeeds, it does not return.
- The current program is replaced by the new program.
- If it returns, an error occurred.

Distinguishing the exec Variants

All exec functions replace the current process image with a new program. They differ in how arguments, environment variables and paths are supplied.

Function	Arguments	Uses PATH?	Environment
execl()	List	No	Inherited
execv()	Vector	No	Inherited
execlp()	List	Yes	Inherited
execvp()	Vector	Yes	Inherited
execle()	List	No	Explicit
execve()	Vector	No	Explicit

- "l": arguments are given as a list:
  execl("/bin/ls", "ls", "-l", NULL);
- "v": arguments are given as a vector:
  *char *args[] = "ls", "-l", NULL;*
  execv("/bin/ls", args);
- "p": the program is searched in PATH:
  execlp("ls", "ls", "-l", NULL);
- "e": a custom environment is provided.

Using `fork()` and `exec()` together

A common pattern is:

1. The parent calls `fork()`.
2. The child calls `exec()`.
3. The parent waits for the child.

Example:

```c
pid_t pid = fork();

if (pid == -1) {
    perror("fork");
} else if (pid == 0) {
    execlp("ls", "ls", "-l", NULL);
    perror("execlp");
    _exit(1);
} else {
    printf("Parent waiting for child %d\n", pid);
}
```

The child process first starts as a copy of the parent, then `exec()` replaces it with the `ls` program.

wait()

The function `wait()` waits until one child process terminates.

```c
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait(int *status);
```

Example:

```c
int status;
pid_t child = wait(&status);

if (child == -1) {
    perror("wait");
} else {
    printf("Child %d finished\n", child);
}
```

The `status` variable contains information about how the child finished.

Interpreting child termination status

Useful status macros

`WIFEXITED(status)` True if the child terminated normally, for example by calling `exit()` or returning from `main()`.

`WEXITSTATUS(status)` Returns the child exit code. Use only if `WIFEXITED(status)` is true.

`WIFSIGNALED(status)` True if the child was terminated by an uncaught signal.

`WTERMSIG(status)` Returns the signal number that terminated the child. Use only if `WIFSIGNALED(status)` is true.

Example:

```
1     int status;
2     waitpid(pid, &status, 0);
3
4     if (WIFEXITED(status)) {
5         printf("Exit code: %d\n", WEXITSTATUS(status));
6     } else if (WIFSIGNALED(status)) {
7         printf("Killed by signal: %d\n", WTERMSIG(status));
8     }
```

waitpid()

`waitpid()` gives more control than `wait()`.

```c
#include <sys/types.h>
#include <sys/wait.h>

pid_t waitpid(pid_t pid, int *status, int options);
```

Example:

```c
int status;

pid_t result = waitpid(pid, &status, 0);

if (result == -1) {
    perror("waitpid");
} else if (WIFEXITED(status)) {
    printf("Child exited with code %d\n",
           WEXITSTATUS(status));
}
```

The first argument controls which child to wait for:

- `pid > 0`: wait for a specific child.
- `pid == -1`: wait for any child, like `wait()`.
- `pid == 0`: wait for any child in the same process group.

Complete Example

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child: PID = %d, PPID = %d\n",
               getpid(), getppid());

        execlp("ls", "ls", "-l", NULL);

        perror("execlp");
        _exit(1);
    }

    int status;
    waitpid(pid, &status, 0);

```

Summary

- `system()` runs a shell command.
- `fork()` creates a child process.
- `exec()` replaces the current program with another one.
- `getpid()` gives the current process ID.
- `getppid()` gives the parent process ID.
- `wait()` waits for any child process.
- `waitpid()` waits for a specific child or group of children.

A common Unix/Linux process pattern is:

`fork()` $\rightarrow$ child calls `exec()` $\rightarrow$ parent calls `waitpid()`

Bibliography

Bibliography references for Linux process-related calls:

1. Linux man-pages project
   system(3) — Linux manual page
   C library function for executing a shell command.
   https://man7.org/linux/man-pages/man3/system.3.html
   fork(2) — Linux manual page
   System call for creating a child process.
   https://man7.org/linux/man-pages/man2/fork.2.html
   exec(3) — Linux manual page
   The exec family of functions: execl, execlp, execle, execv, execvp, execvpe.
   https://man7.org/linux/man-pages/man3/exec.3.html
   execve(2) — Linux manual page
   System call that executes a new program.
   https://man7.org/linux/man-pages/man2/execve.2.html
   wait(2) — Linux manual page
   System calls wait, waitpid, and waitid for waiting for process state changes.
   https://man7.org/linux/man-pages/man2/wait.2.html
2. The Open Group
   system — The Open Group Base Specifications.
   POSIX specification for system().
   https://pubs.opengroup.org/onlinepubs/9699919799/functions/system.html
   fork — The Open Group Base Specifications.
   POSIX specification for fork().
   https://pubs.opengroup.org/onlinepubs/9699919799/functions/fork.html
   exec — The Open Group Base Specifications.
   POSIX specification for the exec family of functions.
   https://pubs.opengroup.org/onlinepubs/9699919799/functions/exec.html
   wait — The Open Group Base Specifications.
   POSIX specification for wait().
   https://pubs.opengroup.org/onlinepubs/9699919799/functions/wait.html
   waitpid — The Open Group Base Specifications.
   POSIX specification for waitpid().
   https://pubs.opengroup.org/onlinepubs/9699919799/functions/waitpid.html
3. W. Richard Stevens and Stephen A. Rago, Advanced Programming in the UNIX Environment., 3rd edition., Addison-Wesley, 2013., ISBN: 978-0321637734.